第1回 演習・宿題

開発環境を整え、LLMを使おう

松尾・岩澤研究室
Matsuo-Iwasawa Lab UTokyo

2026/10/19　演習：佐藤良明

自分の fork を Codespaces で開き、Gemini API と GitHub Copilot を使える状態にして、PR で提出する

許諾なく撮影や第三者への開示を禁止します

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

この回のゴール

以降の回で使う開発環境と、提出の流れを、この回でそろえる。

1

自分の fork を Codespaces で開き、Gemini API を呼び出せる

2

GitHub Copilot でコードを少し直せる

3

作業ブランチから PR を出して提出できる（以降の回もこの流れで提出する）

使うもの：GitHub Codespaces（ブラウザで開く開発環境）、Gemini API（Google の LLM）、GitHub Copilot（AI コーディングツール）。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

準備するもの：個人アカウント２つ

宿題の提出や演習の実施にあたり、GitHubとGoogleのアカウントを利用します。

GitHub の個人アカウント

使うもの：fork、Codespaces、Copilot、PR

> Githubのアカウントを作成することで、Codespacesという仮想環境とCopilotというエージェントも利用できるようになります。

Google の個人アカウント

使うもの：Google AI Studio（Gemini API キーの発行）

> LLMのAPIを使った演習が行えるようになります。

どちらも個人アカウントで用意する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習 45 分の流れ

fork から Codespaces の停止まで、8 つの手順を順に行う。

1	講座 repo を fork する
2	Gemini API キーを発行する
3	Codespaces secret に登録する
4	Codespaces を起動する
5	素のLLMに質問する
6	Copilot でトークン数を表示させる
7	ブランチを作って PR を出す
8	Codespaces を停止する

1〜4　環境を整える

5〜8　LLM と Copilot を使い、提出して片付ける

順番に注意：手順 3 の secret は、手順 4 で Codespaces を起動する前に登録する。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 1　講座 repo を fork する

講座 repo の右上の Fork から、自分のアカウントに fork を作る。

1. 講座 repo を開く：
   https://github.com/matsuolab/lecture-ai-engineering-2026
2. 右上の **Fork** を押す
3. **Owner** に自分のアカウントを選び、**Create fork** を押す
4. 自分のアカウントの下に fork ができたことを確認する



以降の作業はすべて自分の fork で行う。講座 repo に PR を出さない。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 2　Gemini API キーを発行する

Google AI Studio で API キーを作成し、コピーする。キーは手順 3 で使う。

1. Google の個人アカウントで Google AI Studio を開く：
   https://aistudio.google.com/apikey
2. API キーを作成する
3. 作成したキーをコピーする（そのまま手順 3 で登録する）

キーを貼らない場所

- コード
- チャット
- コミット

APIキーは秘密情報です。コードやチャット、コミットに貼らないでください。
漏れた場合は AI Studio でキーを削除し、作り直してください。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

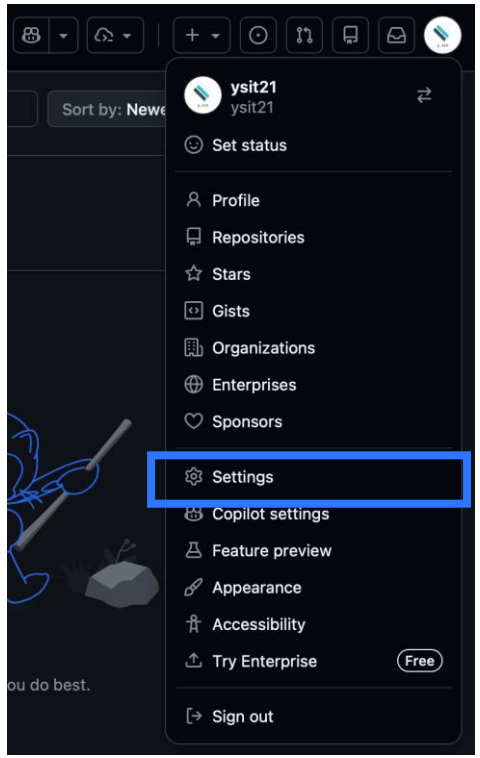
手順 3　Codespaces secret に登録する

API キーはコードに書かず、Codespaces secret として登録する。

右上のプロフィール画像 → **Settings** → **Codespaces** → **Secrets** → **New secret**

GitHub の Settingsと、 forkしたレポジトリの Settingsがあり間違えやすいので注意して下さい。
ここでは、GitHub の Settingsから設定する必要があります。

項目	登録する内容
Name	GEMINI_API_KEY
Value	手順2のキー
Repository access	自分の fork を選ぶ



Codespaces を起動する前に登録してください。起動後に登録した場合は、Codespaces の再起動が必要です。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

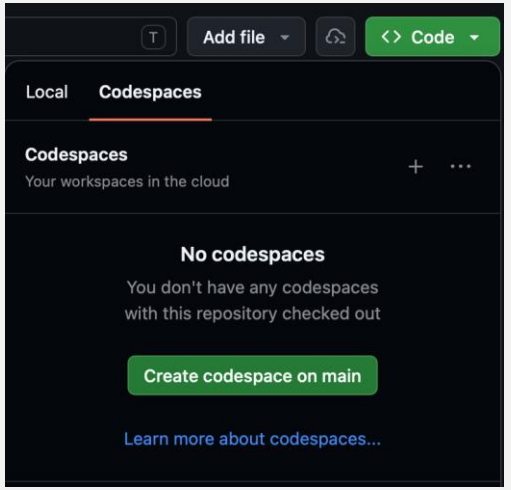
手順 4　Codespaces を起動する

自分の fork で Codespaces を作成し、ターミナルでテストが通ることを確認する。

1. 自分の fork のページで **Code** を押す
2. **Codespaces** を選び、**Create codespace on main** を押す
3. 初回は環境の準備に数分かかる
4. ターミナルで次を実行し、テストが通ることを確認する

```
uv run pytest -q
```

自分の fork のページ



GitHub上のスクリーンショット

マシンは 2-core を推奨（デフォルト値）：コア数が多いマシンほど無料枠を早く使う。無料枠と停止は手順 8。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 5　LLMに質問する

一般的な用語と、講座に固有のことを質問し、答え方の違いを見る。

```
uv run python -m app.ask "LoRAとは何ですか？"
```

一般的な用語

おおむね正しい答えが返る

```
uv run python -m app.ask "AIE2026の講座の宿題にある「3レベル」とは何ですか？"
```

講座に固有の「宿題の3レベル」

素のLLMは講座のことを知らないため、「分からない」と答えるか、一般論でもっともらしく答える

正しい内容は data/glossary.json の homework-levels にある。モデルの答えと比べる。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 5　観察するポイント

同じ質問でも実行ごとに答えが変わる。知らないことでも確信を持って答えることがある。

余裕があれば、ほかの講座に固有の質問も試す

```
uv run python -m app.ask "AIE2026の講座で使う「lecNN タグ」とは何ですか？"
uv run python -m app.ask "AIE2026の第2回では何を学びますか？"
```

一般的な用語	講座に固有のこと	繰り返したとき
おおむね正しく答える	知らないので「分からない」か、一般論でもっともらしく答える	実行ごとに答えが変わる。確信を持って間違えることがある

正しい内容は data/glossary.json の homework-levels にある。モデルの答えと比べる。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 6　Copilot でトークン数を表示させる

app/ask.py の TODO(第1回 演習) の場所で、入力トークン数と出力トークン数を表示するよう改修する。

1. **app/ask.py** を開き、**TODO(第1回 演習)** の場所を探す
2. 入力トークン数と出力トークン数を表示するよう改修する。Copilot の補完やチャットを使ってよい
3. 提案されたコードは、読んで理解してから採用する

GitHub Copilot

使える機能：補完、チャット、エージェント

Copilot Free：コード補完は月 2,000 回まで

学生：認証済みの学生は Copilot Student を無料で使える

AGENT.md / CLAUDE.mdについて

コーディングエージェントには、暗黙知を伝えるための仕組みがあります（第3回でも取り扱います）。巻末の補助資料を参照下さい。

提案されたコードは、読んで理解してから採用する。

出典：docs.github.com　https://docs.github.com/en/copilot/get-started/plans

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 7　ブランチを作って PR を出す

作業ブランチ lec01 から、自分の fork の main へ PR を出す。以降の回もこの流れで提出する。

```
git switch -c lec01
# lectures/01/answer.md を書く
git add -A
git commit -m "第1回 演習"
git push -u origin lec01
```



提出物：lectures/01/answer.md を含む PR。GitHub の自分の fork で、lec01 → main の PR を作成する。

base repository が自分の fork になっていることを確認してください（講座 repo に PR を出さないこと）。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 8　Codespaces を停止する

起動したままだと無料枠を消費し続ける。使い終わったら必ず停止する。

左下の **Codespaces** → **Stop Current Codespace**

Codespaces の無料枠（個人アカウント、毎月）

プラン	ストレージ	コンピュート
GitHub Free	15 GB-month	120 時間
GitHub Pro	20 GB-month	180 時間

倍率：マシンの種類ごとにある（2-core は 2、4-core は 4）。コア数が多いほど無料枠を早く使うので、2-core を選ぶ

使い切ったら：支払い方法を登録していなければ、翌月のリセットまで使えない（課金はされない）

出典：docs.github.com　https://docs.github.com/en/billing/concepts/product-billing/github-codespaces

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

宿題：どれか 1 つで完了

基礎・実装・発展の 3 つのレベルから、1 つを選んで取り組む。

レベル	内容
基礎	同じ質問を3回ずつ、モデルを変えて（AIE_MODEL 環境変数）、または temperature を変えて実行し、回答の違いを answer.md にまとめる
実装	app/ask.py を改修し、会話履歴を保ったまま続けて質問できるようにする
発展	応答時間とトークン数を計測し、複数のモデルや質問で比較する。無料枠のレート制限との関係も考察する

モデルと temperature：切り替え方と注意は次のスライド

提出：lectures/01/answer.md に書き、PR で出す。ルールはその次のスライド

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

宿題の提出とルール

answer.md に書き、演習と同じ流れの PR で提出する。どれか 1 つで完了。

1. **自分の fork の作業ブランチで作業する。**実装・発展でコードを変えた場合は、そのコードも commit
2. **lectures/01/answer.md に書く。**選んだレベル、やったこと、結果、使ったモデル
3. **commit・push して PR を出す。**base repository が自分の fork であることを確認

評価：どれか 1 つ以上は実施して下さい

AI ツール：使ってよい。自分で確かめたことを書く

API キー：answer.md、コード、ログ、スクリーンショットに書かない

うまくいかなかった結果や、差が出なかった結果も、そのまま書く

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

モデルの切り替えと注意

AIE_MODEL 環境変数でモデルを切り替える。どのモデルで実行したかを記録する。

```
AIE_MODEL=gemini-3.8-flash uv run python -m app.ask "LoRAとは何ですか？"
AIE_MODEL=gemini-3.5-flash-lite uv run python -m app.ask "LoRAとは何ですか？"
```

モデル	無料枠の上限（講座の実測）	向いている使い方
gemini-3.5-flash-lite	毎分 15、1 日 500 リクエスト	上限が大きい。繰り返し実行して比べる
gemini-3.8-flash（上位）	毎分 5、1 日 10〜20 前後。503 が返ることも	数問だけ強いモデルで試す

使わない：gemini-3.5-flash（lite なし）。2026-10-08 に非推奨になり、gemini-3.6-flash へ自動転送される

temperature：generate_content の config 引数を調べる。2026-07-21 に Google が非推奨にしたため、無視やエラーになりうる。結果をそのまま記録

上限はモデルによって変わる：AI Studio で現在の上限を確認。利用の上限（クオータ）を確認し、APIを利用できる状態なのか確認する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

第2回の予告：基盤モデルの仕組み

演習は Colab で LoRA を使い、回答の形式を学習して比較する。

講義

- 基盤モデルの仕組み
- 生成設定と出力の揺れ
- LoRA と QLoRA の仕組み

演習

- Colabで LoRA の学習を行う
- 学習の前後で、形式と内容を分けて比べる

今回観察したこと	第2回で扱うこと
同じ質問でも、実行ごとに答えが変わる	生成設定と出力の揺れ
講座のことを知らなくても、もっともらしく答える	モデルが学習で何を身につけ、何を身につけないか

今回観察した「揺れ」と「知らないことを答える」が、第2回の仕組みの話につながる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

まとめ：今日の到達点と次回

以降の回で使う開発環境と、提出の流れを説明しました。

1

自分の fork を Codespaces で開き、Gemini API を呼び出した

2

Copilot を使い、トークン数を表示するようコードを直した

3

lec01 から自分の fork の main へ PR を出した

宿題：基礎・実装・発展のどれか1つで完了。
締切は原則1週間（10/26）

第2回：基盤モデルの仕組み。Colab で LoRA を使う

Codespaces は使い終わったら停止する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

補助資料

再利用可能なコンテキスト（AGENTS.md / CLAUDE.md）

毎回ゼロから読むエージェントに、推測できないことを書いて渡す

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

なぜコンテキストファイルが必要か

エージェントは毎回ゼロから読む。リポジトリの暗黙知を書き出し、同じ前提を毎回渡す。

書いていないと

- 起動やテストのコマンドを推測する
- 既定と違う規約や、つまずきやすい点を知らない
- 毎回、依頼文で同じ説明を繰り返す

→

書いておくと

- 作業の最初に、ファイルから読み込まれる
- リポジトリに commit され、チームで共有・レビューできる
- 依頼文は、その作業に固有のことに絞れる

AGENTS.md

“a README for agents”。必須の項目はなく、普通の Markdownファイル。多くのツールが読む共通の形式

CLAUDE.md

Claude Code が起動時に読む。強制される設定ではなく context として利用されるファイル。必ず実行させたいことは hook という別の仕組みを利用する

出典）agents.md「AGENTS.md」、Claude Code Docs「How Claude remembers your project」

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

主なファイルと読まれ方

共通の AGENTS.md と、ツールごとのファイル。どれをどう読むかはツールが決める。

ファイル	主なツール	読まれ方
AGENTS.md	Codex、Cursor、GitHub Copilot ほか	入れ子にでき、最も近いものが優先。Codex はルートから作業ディレクトリまで連結（既定 32 KiB まで）
CLAUDE.md	Claude Code	上位ディレクトリの分を起動時に連結。サブディレクトリの分は、そこのファイルに触れたときに読む。CLAUDE.md がなければ AGENTS.md を読む
.github/copilot-instructions.md .github/instructions/*.instructions.md	GitHub Copilot	リポジトリ全体／applyTo でパスを指定。cloud agent は AGENTS.md や、ルートの CLAUDE.md・GEMINI.md も使う
.cursor/rules の .mdc	Cursor	description・globs・alwaysApply で読む条件を指定。AGENTS.md にも対応
GEMINI.md	Gemini CLI	グローバル → ワークスペース → アクセスしたディレクトリの順に連結。設定で AGENTS.md も読める

出典）agents.md、OpenAI「Custom instructions with AGENTS.md」、Claude Code Docs「How Claude remembers your project」、GitHub Docs「Adding repository custom instructions」、Cursor Docs「Rules」、Gemini CLI Docs「GEMINI.md」

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

最小構成：入れるもの・入れないもの

各行に「消したら間違えるか」を問う。目安は1ファイル200行未満。

入れるもの

- 推測できない起動・テスト・lint のコマンド
- ディレクトリ構成の要点と、既定と違う規約
- やってはいけないこと
- 完了の定義（どのテストが通れば終わりか）
- リポジトリの作法（ブランチ名、PR の規約）

入れないもの

- 秘密情報（API キーなど）
- コードを読めば分かること、一般的な言語規約
- 長い説明、詳細な API ドキュメント
- 頻繁に変わる情報
- 「clean code を書け」のような自明な指示

200行未満
長いほど context を使い、指示を守る率が下がる

@path で import
再帰は最大4段。import しても context のコストは減らない

強制はされない
必ず実行させたいことは hook で設定する

出典）Claude Code Docs「How Claude remembers your project」「Best practices for Claude Code」

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

常に読む指示と、作業に応じて読む Skill

常に読む指示は短く保つ。作業ごとの手順は Skill に分け、使うときだけ読み込む。

常に読む

CLAUDE.md / AGENTS.md

毎回 context に入る。だから短く、全作業に共通することだけ

作業に応じて読む

Skill：.claude/skills/<name>/SKILL.md

常に入るのは description だけ。本文は使うときに読み込む（progressive disclosure）。

SKILL.md は500行未満が目安

Discovery	→	Activation	→	Execution
description で知る		使うときに本文を読む		手順に沿って実行

一般的なリポジトリでの分け方

ルートの AGENTS.md / CLAUDE.md

常に読む指示：推測できないコマンド、既定と違う規約、テストの手順

.claude/skills/<名前>/SKILL.md

作業ごとの手順（公式ガイドの例：API の規約、Issue を修正する手順）

分ける目安：全作業に要るか、特定の作業だけで要るか

Agent Skills：2025-12 にオープン標準として公開。Codex、GitHub Copilot、Cursor、Gemini CLI なども対応。

出典）Claude Code Docs「Skills」「Best practices for Claude Code」、Anthropic「Introducing Agent Skills」（2025-10-16、2025-12-18 追記）、agentskills.io「Agent Skills Overview」

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

効果のエビデンス：書けば良くなるとは限らない

LLM が生成したファイルは成功率を下げ、開発者が書いたものは上げた。ただしコストは増える。

約 −3%
LLM が生成した context file：タスク成功率

約 +4%
開発者が書いた context file：タスク成功率

20%超
推論コストの増加

Gloaguen ら（ETH Zurich ほか）。AGENTbench（12リポジトリ・138件）と SWE-bench Lite で、3つのコーディングエージェントを評価。指示には従うが、リポジトリ概要の記述は役に立たず、非標準の慣習を伝えるのには有用だった。

> **別の研究（Lulla ら）**：AGENTS.md があると、実行時間の中央値が28.64%短く、出力トークンが16.58%少ない。タスク完了は同程度（10リポジトリ・124件の PR）。

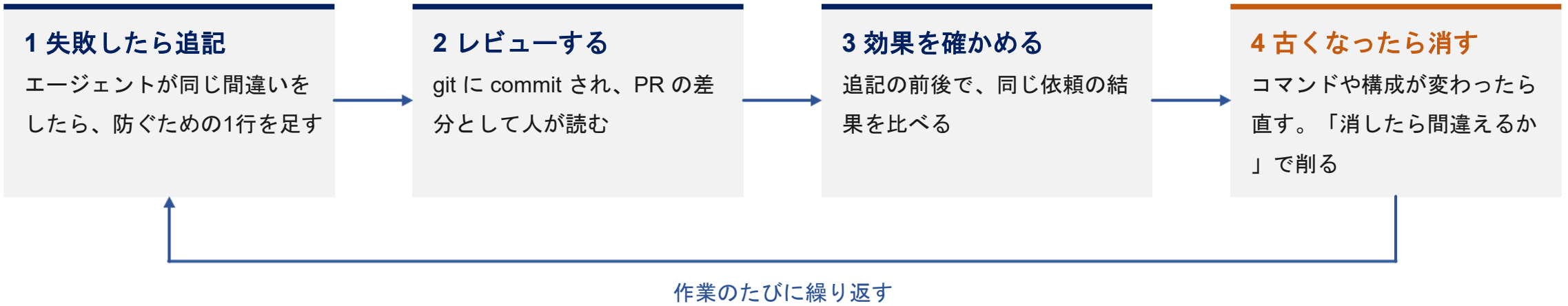
自動生成して終わりにしない：短く・人が書き・効果を確かめる

出典）Gloaguen ら「Evaluating AGENTS.md」arXiv:2602.11988（2026-02-12、v3 2026-09-29）、Lulla ら「On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents」arXiv:2601.20404（2026-01-28）。いずれもプレプリント

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

書いて終わりにしない

コンテキストファイルもコードと同じ。commit し、レビューし、育てる。



公式ガイド：CLAUDE.md は git に commit し、コードと同じように見直して削る。

効果の見方：変更のあと、エージェントの振る舞いが実際に変わったかを観察する。

出典）Claude Code Docs「Best practices for Claude Code」

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO